Billium: A Peer-to-Peer Mining Game Chain and Non-Custodial Solana Wallet

The Billium Developers

White Paper, Version 1.0 (draft for review), October 2026

Abstract. A purely peer-to-peer mining game would let anyone with an ordinary computer compete for token rewards without relying on a game server. We propose a lightweight proof-of-work chain, the Billium Game Chain, that records blocks and reward claims but never value transfers, paired with a non-custodial Solana wallet and a payout service that verifies blocks independently and settles rewards as the BILLIUM SPL token. Miners hash block headers with double SHA-256 against a deliberately modest target, so that a four-core office computer can find a block alone in roughly nine to eighteen minutes at launch, and a difficulty ceiling keeps the network from becoming too hard for such machines. Rewards begin at 1,125 BILLIUM per block, halve every 200,000 blocks, and are capped at 450,000,000 of a fixed supply of 1,000,000,000. The chain needs no central coordinator, but settlement does: players rely on a known operator to honor valid claims. We state that limitation openly, along with the fact that proof-of-work still favors greater hashpower.

1. Introduction

Token mining games tend to fail in the same ways. Hardware arms races make low-end computers irrelevant. Wallet setup is confusing and network fees are unclear. Reward distribution is opaque and payouts are late. Peer-to-peer synchronization is fragile, and the tokenomics are hard for a player to verify. What is needed is a game whose rules can be inspected, whose hardware requirement is an ordinary desktop, and whose trust assumptions are stated plainly.

We separate the problem into three parts. A *game chain* provides a public, replicable record of who found which block, using a Bitcoin-style proof-of-work design [1] with no smart contracts and no spendable fees; the block subsidy is its only incentive. *Solana* holds the BILLIUM token and settles every transfer, using the standard SPL Token program [2]. A *payout service* connects the two: it observes the game chain, independently verifies each claimed block, waits for confirmations, and pays the winner on Solana. Players keep their own private keys throughout. Figure 1 shows how the parts fit together.

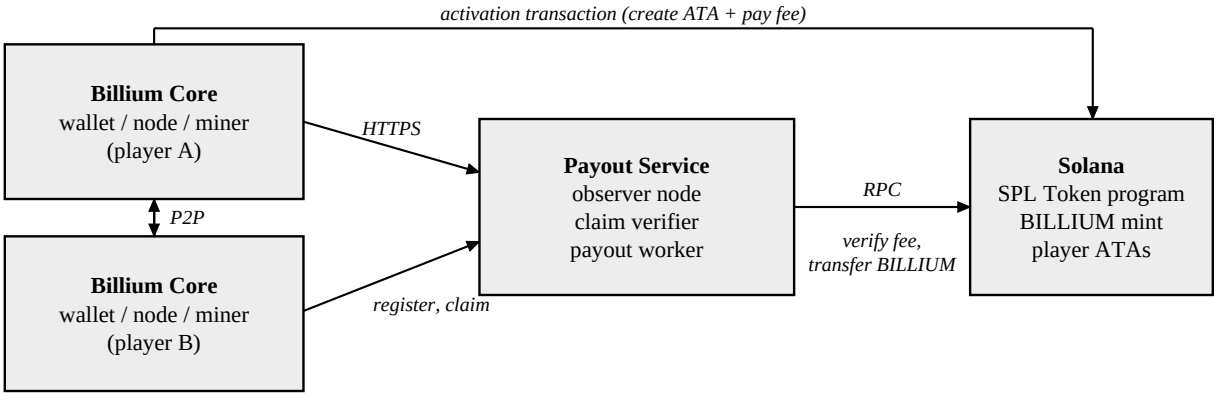

Figure 1. System overview. The game chain is peer-to-peer; reward settlement is operated by a known service.

Billium is not a general-purpose smart-contract platform and does not replace Solana. It is a game whose rewards happen to be settled on one. Four principles guide the design: a low hardware barrier,

transparent rewards, a non-custodial wallet, and an explicit trust model.

2. Wallet and Activation

Billium Core imports an existing Solana key from a Solana CLI keypair file, a base58 string, or a Sollet-style JSON keypair. The key is never sent to a server. It is stored locally under a password using PBKDF2-HMAC-SHA256 with 210,000 iterations and a random salt [3], and encrypted with AES-256-GCM [4]. The wallet locks after inactivity and must be unlocked explicitly before mining or sending. There is no custodial recovery: a lost key is lost.

To mine, a player activates the wallet once. The application builds a single atomic transaction that (a) idempotently creates the player's associated token account (ATA) for BILLIUM and (b) transfers a fixed activation fee to the operator's admin wallet. Because both instructions travel in one transaction, either both are committed or neither is. The exact cost is displayed before signing.

Item	Lamports	SOL
ATA rent (queried dynamically for 165 bytes)	2,039,280	0.00203928
Admin fee (one-time, non-refundable)	4,080,000	0.00408000
Network transaction fee (estimate)	5,000	0.00000500
Estimated total	**6,124,280**	**0.00612428**

Table 1. Activation cost. If the ATA already exists, the cost is about 4,085,000 lamports.

The wallet then registers with the payout service by submitting its address, ATA, the activation transaction signature, and a signed challenge nonce. The service checks that the signature is valid, that the transaction is confirmed and error-free, that it moved at least the required fee from the player to the admin wallet, that the ATA exists with the correct mint and is not frozen, and that neither the wallet, the transaction, nor the nonce has been used before. Only then is the wallet marked *eligible*. The fee covers development and payout infrastructure. It is not a token purchase and does not guarantee any reward.

Mining is enabled only while the wallet is unlocked, the ATA is valid, the wallet is eligible, the payout service is reachable, and the local chain is synchronized. Eligibility is rechecked about every ten minutes. If the service becomes unreachable, mining pauses so that no reward is produced that cannot be settled.

3. Game Chain

Each block consists of an 80-byte header, a coinbase, and a miner signature (Figure 2). The header layout follows Bitcoin: a 4-byte version, the 32-byte hash of the previous block, a 32-byte Merkle root, a 4-byte Unix timestamp, a 4-byte compact target (*bits*), and a 4-byte nonce. The Merkle root commits to the coinbase, which records the block height, the miner's Ed25519 public key [5], the reward, and a short message. The miner then signs the block hash with the same key. This binds the reward to the wallet that actually found the block, so a block cannot be claimed by someone who merely observed it on the network.

The first block, the genesis block, is created once by the developers and distributed as a file. Its hash, together with the chain identifier, is hardcoded into both the application and the payout service, so that every honest participant agrees on which chain is Billium. Genesis fixes the initial target, the 600-second target block time, the 20-block retarget interval, the 200,000-block halving interval, the initial reward, and the token mint. Nodes store the chain locally in SQLite and can rebuild it from

peers if it is corrupted.

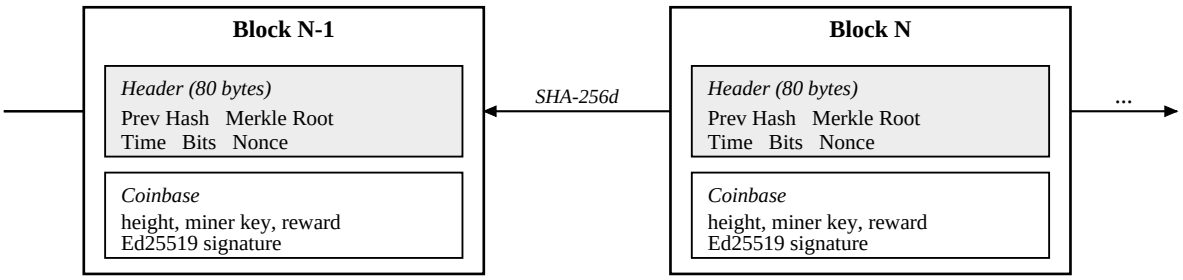

Figure 2. Blocks are chained by the hash of the previous header; the coinbase carries the reward recipient.

4. Proof-of-Work

To timestamp blocks on a peer-to-peer basis we use a proof-of-work system in the spirit of Hashcash [6]. The block hash is the SHA-256 function applied twice to the header [7]. A block is valid when this 256-bit value, read as a number, does not exceed the target decoded from *bits*. Miners increment the nonce; when the 32-bit nonce space is exhausted they change an extra-nonce field in the coinbase and rebuild the Merkle root.

Nodes accept the chain with the greatest cumulative proof-of-work. A block is valid only if it extends a known block, meets its target, carries a correct signature and coinbase commitment, has a timestamp above the median of the previous 11 blocks and no more than two hours in the future, pays exactly the subsidy for its height, and uses the difficulty required by the current retarget window. If a competing chain gains more cumulative work, a node reorganizes to it. Blocks left behind are marked orphaned and are never paid.

Difficulty is adjusted every 20 blocks. If the previous window took t seconds against an expected $20 \times 600 = 12{,}000$, the new target is

$$T_{\text{new}} = T_{\text{old}} \times \text{clamp}(\, t \,/\, 12{,}000,\ 1/4,\ 4\,), \quad 2^{220} \le T \le 2^{226}$$

so that no single retarget moves difficulty by more than a factor of four. The easiest allowed target is the initial one, 2^{226}. The hardest, 2^{220}, is a deliberate *difficulty ceiling*: it prevents the network from becoming so hard that low-end players have no realistic chance. If total hashrate outgrows what the ceiling implies, blocks simply arrive faster than ten minutes. We accept that trade-off in exchange for accessibility.

Parameter	Value	Parameter	Value
Block time target	600 s	Initial reward	1,125 BILLIUM
Header size	80 bytes	Halving interval	200,000 blocks
Proof-of-work	SHA-256d	Mining allocation cap	450,000,000
Initial target	2^{226}	Payout confirmations	6
Hardest target	2^{220}	Mining threads	2 (max 4)
Retarget interval	20 blocks	Peer limit	16
Retarget clamp	4x	Activation fee	4,080,000 lamports

Table 2. Core protocol parameters.

5. Network

The steps to run the network are as follows:

1) A node that finds a valid block announces and relays it to its peers.
2) Each node validates the block independently and discards it if any rule in Section 4 fails.
3) Nodes that accept the block relay it onward and restart their miners on the new tip.
4) If two blocks are found at once, nodes keep both and work on the first one received; the tie breaks when one branch gains more cumulative work, and nodes switch to it.

Peers speak length-prefixed JSON over TCP. Two nodes open a connection with a handshake carrying the protocol version, chain identifier, genesis hash, height, tip hash, and cumulative work, so that a node on a different chain is rejected immediately. The protocol has thirteen message types for version negotiation, header and block download, inventory announcement, address gossip, keep-alive, and rejection. Peers are found through configured seed nodes, address gossip, and manual entry, with a default limit of 16 connections. Peers that send invalid blocks or spam are scored down and banned. The payout service also runs as a seed node. Synchronization downloads headers first and then blocks, so that a new node on a local network can catch up on 10,000 blocks in under a minute.

6. Accessibility on Low-Spec Hardware

The reference machine is a four-core 2.0 GHz CPU with 4 GB of RAM, integrated graphics, and Java 17. The application targets a heap below 512 MB and mines with two threads by default, configurable from one to four. It needs no GPU and no native code, and mining stops within one second when the wallet locks or the service disappears.

With target T and a local hashrate H, the expected time for a solo miner to find a block is

$$E[t] = (2^{256} / T) / H \quad (2^{30} \text{ hashes at } T = 2^{226})$$

Local hashrate	**1.0 MH/s**	**1.5 MH/s**	**2.0 MH/s**	**4.0 MH/s**
Expected time per block	17.9 min	11.9 min	8.9 min	4.5 min

Table 3. Expected solo time per block at the initial target.

Block discovery is memoryless, so the chance of finding at least one block within time t is $1 - e^{-t/E}$. At 1 MH/s that is about 96% within an hour, but these are expectations rather than guarantees, and solo variance is high.

As players join, the retarget raises difficulty to hold the ten-minute average, and a player's share of blocks equals their share of network hashrate. The ceiling of 2^{220} corresponds to 2^{36} hashes per block, or about 114.5 MH/s of network hashrate at ten-minute blocks. Among 100 players of 1 MH/s each, which is the initial player target, the network stays below that ceiling and each player expects about 1% of the 144 daily blocks, roughly 1.4 blocks per day.

Limits of fairness. Global proof-of-work is not egalitarian. A player with four times the hashrate earns four times as many blocks on average. The two-thread default blunts the advantage of many-core machines, but it is a client-side cap and a modified client can bypass it. Billium is accessible to low-spec hardware; it is not hashpower-equal. If strict equality becomes a requirement, a future version can add a share-based pool or a lottery mode in which valid shares earn tickets and a deterministic draw selects the block winner, without changing the core chain.

7. Payout

The game chain records who earned a reward; it does not move tokens. The payout service is the only entity that does. It runs the same validation code as a desktop node, so it can check every claim for itself and never trusts a player's reported height or reward. A player submits the block header, coinbase, nonce, and signature. The claim is accepted only if the block lies on the service's best chain and has reached the confirmation depth, six blocks by default, about one hour at ten-minute blocks.

A claim moves through the states *detected*, *confirming*, *confirmed*, *needs ATA*, *paying*, and *paid*, with exceptional states for orphaned blocks, failed payouts, and operator pauses. The block hash is the unique claim key, so a block can never be paid twice. Every attempt is recorded with its transaction signature. After an ambiguous RPC result, the service checks the signature status before retrying; retries use exponential backoff, and repeated failures go to a manual review queue.

Payments come from a hot wallet that holds a limited balance, proposed at one to five million BILLIUM plus SOL for fees, refilled from a developer-controlled treasury under a documented policy. Keys are encrypted at rest and never logged; administrative endpoints require strong authentication and network restriction. The operator commits to publishing its treasury addresses, aggregate payout statistics, a claim status endpoint, and reports of any downtime or failed payouts.

8. Token and Incentive

BILLIUM is a classic SPL token with a fixed supply of 1,000,000,000 and nine decimals, which the application reads from the chain. Mint and freeze authority are to be revoked after the initial allocation. The token is a game reward and utility token. It is not an investment contract, a claim on revenue, or a promise of liquidity or price.

Allocation	Amount	Share	Custody
Mining rewards	450,000,000	45%	Paid out by the payout service
Liquidity	450,000,000	45%	Developer-controlled treasury (v1)
Developer reserve	100,000,000	10%	Developer treasury, no vesting in v1

Table 4. Token allocation.

The block reward starts at 1,125 BILLIUM and halves every 200,000 blocks. For a block at height h the reward is

$$R(h) = 1{,}125 / 2^{\text{floor}(h / 200{,}000)}, \quad \text{total} = 200{,}000 \times 1{,}125 \times (1 + 1/2 + 1/4 + \ldots) = 450{,}000{,}000$$

Epoch	Blocks	Reward	Issued	Cumulative	Duration
0	0 – 199,999	1,125	225,000,000	225,000,000	3.8 yr
1	200,000 – 399,999	562.5	112,500,000	337,500,000	3.8 yr
2	400,000 – 599,999	281.25	56,250,000	393,750,000	3.8 yr
3	600,000 – 799,999	140.625	28,125,000	421,875,000	3.8 yr
4	800,000 – 999,999	70.3125	14,062,500	435,937,500	3.8 yr
5	1,000,000 – 1,199,999	35.15625	7,031,250	442,968,750	3.8 yr

Table 5. Emission schedule (first six epochs of an asymptotic series).

The series approaches the 450,000,000 cap but never reaches it, so the implementation counts in integer base units, caps cumulative issuance, and stops paying once the allocation is spent. Half of the mining allocation is issued in about 3.8 years. Reaching 25% of it, about 1.9 years in, is the proposed trigger for the liquidity phase, which is not automatic and depends on operator readiness, market

conditions, legal review, and treasury custody. The liquidity allocation is not burned.

Milestone	Mined	Approx. block	Approx. time
25% of mining allocation	112,500,000	100,000	1.9 years
50% of mining allocation	225,000,000	200,000	3.8 years
75% of mining allocation	337,500,000	400,000	7.6 years
87.5% of mining allocation	393,750,000	600,000	11.4 years

Table 6. Emission milestones at a ten-minute block time.

9. Security and Trust Model

Players trust the payout operator to settle valid claims, the Solana RPC provider for chain data unless they run their own, and the published genesis hash and application binary. The game chain has a modest security budget and is unsuitable for securing high-value assets. Most other threats are addressed by mechanism rather than trust:

Threat	Mitigation
Private key theft	Encrypted keystore, inactivity lock, no key logging
Forged activation or replayed registration	On-chain check of sender, recipient, amount, and confirmation; unique wallet and fee transaction; nonce
Fake block claim	Independent proof-of-work and chain validation by the service
Double or orphan payout	Unique block hash, idempotent worker, status checks, six confirmations
Hot wallet compromise	Limited balance, treasury separation, key isolation, monitoring
P2P spam and invalid blocks	Peer scoring, rate limits, banning
Fee confusion	Exact cost breakdown shown before signing

Table 7. Principal threats and mitigations.

An attacker with a majority of hashpower can rewrite recent history, and on a small network this is a real risk. Following Nakamoto [1], if honest nodes control a fraction p of the hashrate and the attacker a fraction $q < p$, the probability that the attacker ever overtakes a chain z blocks ahead falls as $(q/p)^z$. For $q = 0.3$ and $z = 6$, this is about 0.6%. This is why payouts wait six confirmations, and why the operator monitors the chain and can pause payouts. Because rewards are modest and settled off-chain, the cost of a successful attack is bounded by what the operator has not yet paid.

Attacker	$z = 0$	$z = 1$	$z = 2$	$z = 3$	$z = 4$	$z = 5$	$z = 6$
$q = 0.1$	1.0000	0.1111	0.0123	0.0014	0.0002	1.7×10^{-5}	1.9×10^{-6}
$q = 0.3$	1.0000	0.4286	0.1837	0.0787	0.0337	0.0145	0.0062
$q = 0.4$	1.0000	0.6667	0.4444	0.2963	0.1975	0.1317	0.0878

Table 8. Probability that an attacker with hashrate fraction q ever catches up from z blocks behind.

The table shows why the confirmation depth is a policy rather than a constant. A lower value improves the player experience but widens exposure to reorganizations, and an operator who observes a sudden rise in hashrate can raise the depth or pause payouts before any claim is settled. An attacker at or above one half of the hashrate succeeds with certainty, and no confirmation depth helps; for that case the safeguards are monitoring and the operator's ability to withhold payment.

10. Requirements and Acceptance

The version 1 client must import and encrypt wallets in all three formats, show SOL and BILLIUM balances, create and validate the ATA, and send BILLIUM with a checked transfer that names the mint and decimals. Sending to a wallet without an ATA is blocked with an explanation, since the sender would otherwise create an account on the recipient's behalf. The node must validate proof-of-work, signatures, timestamps, and subsidy, retarget every 20 blocks, and handle orphans and reorganizations by cumulative work. The payout service must verify claims independently, track confirmations, and pay idempotently by block hash.

We define acceptance by observable behavior. Wrong passwords fail without leaking key material. Activation is atomic, and a wallet cannot register twice or reuse a fee transaction. A valid block is accepted by at least three independent nodes. An orphaned block is not paid, and a duplicate claim returns the existing payout status rather than a second payment. On the reference machine, solo mining at the initial target finds a block within the expected twenty minutes, the interface stays responsive, memory stays below 512 MB over a 24-hour test, and the application never exceeds its configured thread count.

During the beta we will measure success by outcomes: more than 80% of new users completing onboarding, an average block time between 8 and 12 minutes, payout success above 99%, zero double payouts, and more than 99% crash-free sessions. Version 1 deliberately omits general smart contracts, on-chain verification of proof-of-work, token minting inside the app, exchange or liquidity-pool integration, GPU mining, and any guarantee of token price or liquidity.

11. Roadmap and Open Items

Version 1 targets Windows 10 and 11 with a Swing desktop client, a headless Java payout service, SQLite storage, and SolanaJ for transactions. Development is planned over fourteen weeks from 26 October 2026: wallet and Solana integration, game core, peer-to-peer networking, mining and chain interface, payout service, onboarding, end-to-end hardening, and a release candidate, followed by a buffer. Mainnet readiness is expected in February 2027, or March with a two-to-four-week audit funded from accumulated activation fees. A capped public beta will tune difficulty and fees against real telemetry.

Several items remain open: the final public addresses for the mint, admin wallet, treasury, and hot wallet; the audit scope and auditor; a plan to migrate the single developer-controlled treasury to a multisignature wallet before the liquidity phase; and legal review of any jurisdictional restrictions. Pool or lottery fairness, hardware wallet support, and a mobile companion are deferred to later versions. Parameters, fees, and timelines in this draft may change before launch.

12. Conclusion

We have proposed a deliberately scoped mining game: a proof-of-work chain tuned so that an ordinary computer can take part, a non-custodial wallet, a published and capped emission schedule, and a payout service that verifies every block for itself before paying. The chain is peer-to-peer and needs no game server, while settlement is centralized and says so. The central tension is between accessibility and the fairness of proof-of-work. The modest target, the two-thread default, and the difficulty ceiling lower the barrier to entry but cannot make a low-end CPU equal to a high-end miner. If the game outgrows its design target, a pool or lottery mode can restore fairness without changing the chain. Billium makes no claim to be a trustless financial network; its proposition is accessibility, clear rules, and a mining loop that works on the machine a player already owns.

Disclaimer. This is a technical and product design document, not an offer of securities, a solicitation, or financial advice. The value, liquidity, and availability of BILLIUM are not guaranteed, and rewards depend on network conditions and operator policy. Players should understand the software and comply with local law before participating.

References

[1] S. Nakamoto, “Bitcoin: A Peer-to-Peer Electronic Cash System,” 2008.

[2] Solana Foundation, “SPL Token Program,” Solana Program Library documentation.

[3] K. Moriarty, B. Kaliski, A. Rusch, “PKCS #5: Password-Based Cryptography Specification Version 2.1,” RFC 8018, 2017.

[4] M. Dworkin, “Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC,” NIST SP 800-38D, 2007.

[5] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, B.-Y. Yang, “High-speed high-security signatures,” *Journal of Cryptographic Engineering*, 2(2), 2012.

[6] A. Back, “Hashcash - A Denial of Service Counter-Measure,” 2002.

[7] National Institute of Standards and Technology, “Secure Hash Standard (SHS),” FIPS PUB 180-4, 2015.